THOMAS MORE

JavaScript

Objecten

Inhoudsopgave

- Objecten
  - Object literals
  - Prototype
  - for … in
- Build-in objecten
  - Date
  - Math
  - String

JavaScript

Algemene informatie

Wat is een object

- Een object is een **datatype** *(zoals Number, Date, String…)*
- Bijna alles in JavaScript is een **object**
  - Uitzonderingen: *undefined*, *boolean*, *number*, *bigint*, *string*, *symbol*

- We kunnen een **collectie** van **properties** en **methods** bewaren.
- **Eigenschappen** van een object zijn **properties**.
- **Functies** van een object zijn methodes (**methods**).

- Vb: object auto
  - **Properties**: vermogen, kleur,...
  - **Methods**: starten, versnellen,…

Object aanmaken

01_objects.js

- Via een object literal
  - Door dadelijk waarden aan het object toe te kennen.

    const obj = {prop1: val1, prop2: val2, ...}

```
// Een leeg object zonder waarden, kunnen later toegevoegd worden
const emptyObject = {}

// Een object met twee properties.
const person = {
  firstName: 'Alan',
  name: 'Turing',
}
```

Object aanmaken

01_objects.js

- Via een object literal
  - Door dadelijk waarden aan het object toe te kennen.

    const obj = {prop1: val1, prop2: val2, ...}

```
// Een object met vijf properties, waarvan er één een methode is.
const alan = {
  firstName: 'Alan',
  name: 'Turing',
  birthYear: 1912,
  deathYear: 1954,
}
```

This

01_objects.js

- In JS => this is het object dat de code "bezit"
  - We verwijzen naar het **object** zelf
  - hier dus naar de **eigenschappen** van het **object** *alan*.

```
// Een object met vijf properties, waarvan er één een methode is.
const alan = {
  firstName: 'Alan',
  name: 'Turing',
  birthYear: 1912,
  deathYear: 1954,
  age() {
    return this.deathYear - this.birthYear;
  },
}
```

Eigenschappen en methoden

01_objects.js

- Kunnen na initialisatie toegevoeg/aangepast worden

- Aanspreken via
  - Puntnotatie:
    objectnaam.**eigenschap;** en objectnaam.**methode();**
  - Arraynotatie:
    objectnaam**['eigenschap'];** en objectnaam**['methode']();**

- De **eigenschappen** kunnen o.a. volgende types hebben:
  - string, number, boolean, array, object, function, …

Eigenschappen en methoden

01_objects.js

- Aanpassen / toevoegen

```
alan.name = 'TURING';
alan.accomplishments = 'Turing Machine, Enigma Codebreaker';
```

- Uitlezen

```
// Gebruik zoveel mogelijk deze manier om properties/methodes te gebruiken.
console.log(
  `${alan.firstName} ${alan.name} was born in ${alan.birthYear} and died in ${alan.deathYear} at age ${alan.age()}.`
);

// Gebruik de syntax met vierkante haken als de property niet hardcoded is, maar uit een variabele komt.
console.log(
  `${alan['firstName']} ${alan['name']} was born in ${alan['birthYear']} and died in ${alan['deathYear']} at age ${alan['age']()}.`
);
```

Waarden uitlezen

02_complex_objects.js

- Object in object
  - Sub-namespaces

- Opvragen via

```
alan.name.first
alan['name']['first']
alan.name['first']
alan['name'].first
```

```
const alan = {
  name: {
    first: 'Alan',
    last: 'Turing',
  },
  life: {
    birth: 1912,
    death: 1954,
  },
  age() {
    return this.life.death - this.life.birth;
  },
  accomplishments: [
    'Turing Machine',
    'Enigma Codebreaker',
  ],
}
```

Prototype inheritance

03_prototype.js

- Elk object heeft een **prototype** dat de methodes en properties van de parent bevat
- **Object.prototype** is het bovenste object waarvan al de rest overerft
- Alles wat in het prototype zit kan gebruikt worden als property/methode van een object

```
// Voeg twee debugging prototypes toe aan Object.prototype.
// Deze methodes zijn beschikbaar voor alle objecten.
Object.prototype.objectName = 'UNKNOWN';
Object.prototype.debug = function() {
  console.log(`-------------${this.objectName}-------------`);
  console.log(this)
  console.log('-----------------------------------------');
}
```

Prototype inheritance

03_prototype.js

```js
const alan = {
  objectName: 'Alan Turing Object',
  name: {
    first: 'Alan',
    last: 'Turing',
  },
  life: {
    birth: 1912,
    death: 1954,
  },
  age() {
    return this.life.death - this.life.birth;
  },
  accomplishments: [
    'Turing Machine',
    'Enigma Codebreaker',
  ],
}

alan.debug();
```

```
// Uitvoer
-------------Alan Turing object-------------
{
  objectName: "Alan Turing object",
  name: {
    first: "Alan",
    last: "Turing",
  },
  life: {
    birth: 1912,
    death: 1954,
  },
  age: [Function: age],
  accomplishments: [ "Turing Machine", "Enigma Codebreaker" ],
}
--------------------------------------------
```

Prototype inheritance

03_prototype.js

- Elk object heeft een **prototype** dat de methodes en properties van de parent bevat
- **Object.prototype** is het bovenste object waarvan al de rest overerft
- Via **Array.prototype**, **String.prototype**, … kunnen methodes toegevoegd worden aan de standaardobjecten

```
Array.prototype.keepEven = function() {
  return this.filter(i => i % 2 === 0);
}

const numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];

console.log(numbers.keepEven());
```

```
// Uitvoer
[ 2, 4, 6, 8, 10 ]
```

for….in

04_for.js

Syntax:

for (**prop** in **object**) {
prop geeft de eigenschap // firstName
object[**prop**] geeft de waarde van de eigenschap. // Alan
}

```
// Onderstaande lus itereert over alle properties van het object alan.
for (const property in alan) {
  console.log(property + ':', alan[property]);
}
```

```
// Uitvoer
firstName: Alan
name: Turing
birthYear: 1912
deathYear: 1954
age: [Function: age]
objectName: UNKNOWN
debug: [Function]
```

for….in

04_for.js

Syntax:

```
for (prop in object) {
   prop geeft de eigenschap // firstName
   object[prop] geeft de waarde van de eigenschap. // Alan
}
```

*The **problem** with for...in loop is that it iterates through the properties in the prototype chain as well. Since the objects in JavaScript can inherit properties from their prototypes, the for...in statement will loop through those properties as well.*

*To **avoid** iterating over prototype properties while looping an object, you need to **explicitly** check if the property belongs to the object by using the hasOwnProperty() method.*

for….in

04_for.js

Syntax:

for (**prop** in **object**) {
prop geeft de eigenschap // firstName
object[**prop**] geeft de waarde van de eigenschap. // Alan
}

```
// Onderstaande lus itereert over alle properties van het object alan.
for (const property in alan) {
  // Print enkel de properties van dit object, niet diegene in het prototype.
  if (alan.hasOwnProperty(property)) {
    console.log('  ' + property + ':', alan[property]);
  }
}
```

```
// Uitvoer
firstName: Alan
name: Turing
birthYear: 1912
deathYear: 1954
age: [Function: age]
```

for….of en foreach

04_for.js

Syntax:

```
for (const item of Object.entries(object) {
   item geeft het object als array. // ["voornaam", "thomas"]…
}

Object.entries(object).foreach(([key, value]) => {
   value geeft de waarde van de eigenschap. // thomas
   key geeft de waarde van de eigenschap. // voornaam
})
```

Bekijk ook zeker volgende methoden: Object.keys(), Object.values()

for….of en foreach

04_for.js

```
for (const item of Object.entries(alan)) {
  console.log('  ' + item[0] + ':', item[1]);
}

for (const key of Object.keys(alan)) {
  // Object.keys geeft een array terug met
  // alle property namen.
  console.log('  ' + key + ':', alan[key]);
}
```

```
// Uitvoer
firstName: Alan
name: Turing
birthYear: 1912
deathYear: 1954
age: [Function: age]
```

```
for (const value of Object.values(alan)) {
  // Object.values geeft een array terug met
  // alle property waarden.
  // De keys zijn niet beschikbaar in deze array.
  console.log('  ' + value);
}
```

```
// Uitvoer
Alan
Turing
1912
1954
age() { … }
```

Deconstructing

05_deconstructing.js

- Via **deconstructing** kunnen één of meer properties van een **object** eenvoudig bewaard worden in een variabele.
  - Volgorde is **niet** belangrijk
  - Naam is **belangrijk**

```
const alan = {
  firstName: 'Alan',
  name: 'Turing',
  birthYear: 1912,
  deathYear: 1954,
  age() {
    return this.deathYear - this.birthYear;
  },
}
```

```
const { birthYear, deathYear, firstName, name } = alan;
console.log(`${firstName} ${name} was born in ${birthYear} and died in ${deathYear}.`);

// Uitvoer
// Alan Turing was born in 1912 and died in 1954.
```

Deconstructing

05_deconstructing.js

- Via **deconstructing** kunnen één of meer elementen van een **array** eenvoudig bewaard worden in een variabele.
  - Volgorde is **belangrijk**
  - Naam is **niet** belangrijk

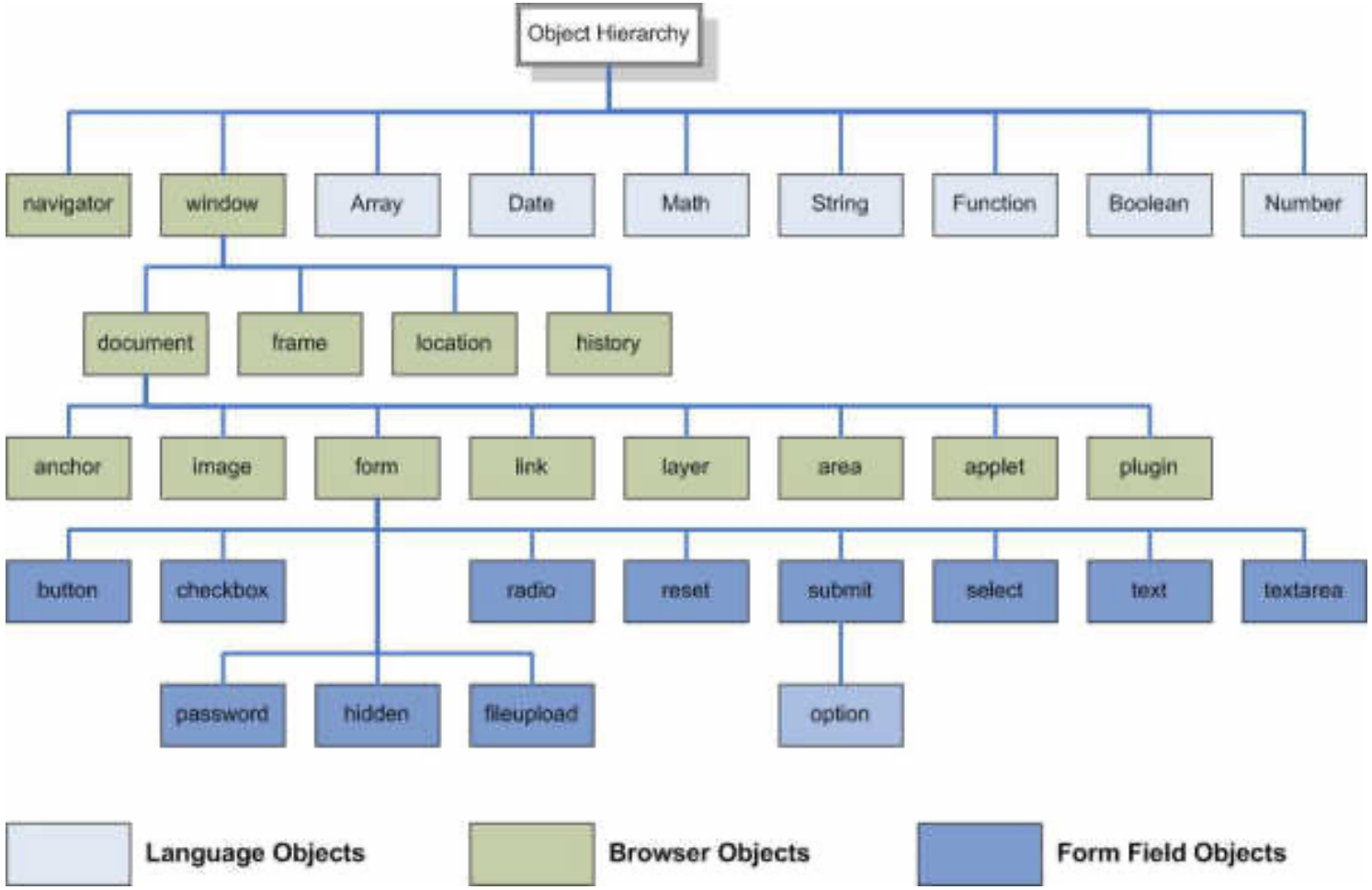
```
const accomplishments = ['Turing machine', 'Enigma code breaking', 'Turing test'];

const [firstAccomplishment, , accomplishment3] = accomplishments;
console.log(firstAccomplishment, accomplishment3);
// Uitvoer: Turing machine Turing test

const [theFirstAccomplishment] = accomplishments;
console.log(theFirstAccomplishment);
// Uitvoer: Turing machine
```

THOMAS MORE

JavaScript

Build-in objects

Object hierarchy



Niet alle objecten zijn in deze afbeelding weergegeven!

Overzicht OBJECTEN

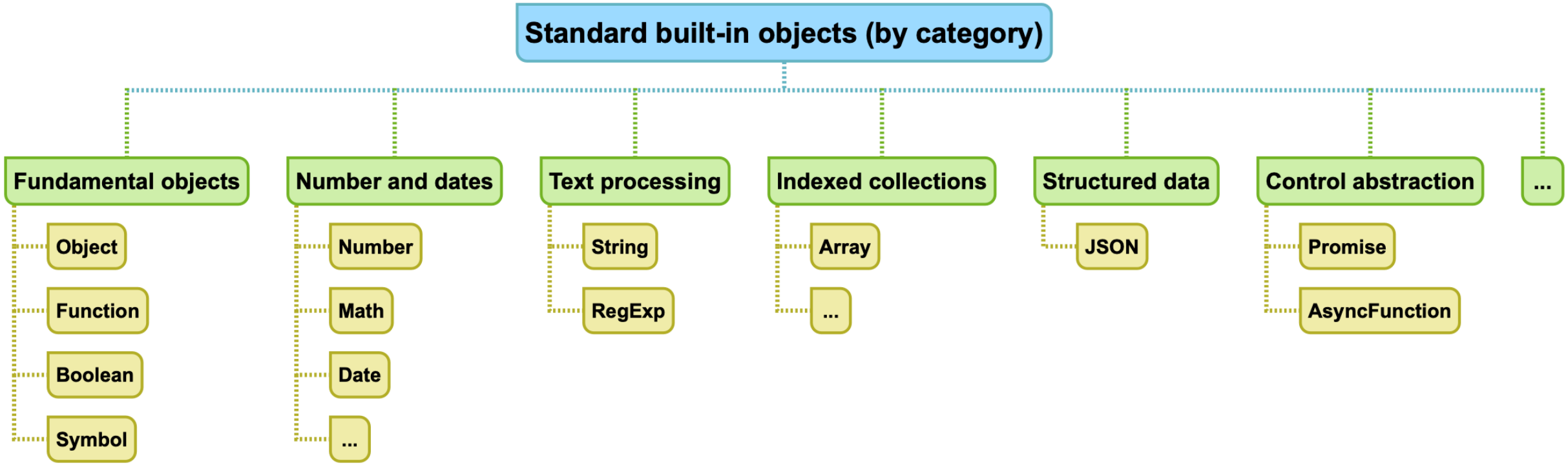
- **JavaScript Built-in objects**
  - Ingebouwde JavaScriptobjecten
- **JavaScript Browser Object Model (BOM)**
  - Objecten specifiek gebonden aan de browser
- **JavaScript Document Object Model (DOM)**
  - Objecten specifiek gebonden aan het document (webpagina)

Overzicht OBJECTEN

- **JavaScript Built-in objects**
  - Ingebouwde JavaScriptobjecten
- **JavaScript Browser Object Model (BOM)**
  - Objecten specifiek gebonden aan de browser
- **JavaScript Document Object Model (DOM)**
  - Objecten specifiek gebonden aan het document (webpagina)

BOM & DOM worden pas besproken in les 4 als we de overstap maken naar de browser.

JS built-in Global Object Model



Volledige referentie: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects

JS built-in Global Object Model

- Niet onder een primair knooppunt
- Behoren tot de specifieke taal (JavaScript)
- Objecten voor het werken met
  - Datums
  - Wiskundige gegevens
  - Data (gegevens)
  - Errors
  - …

Date object

06_date.js

- **Datum en tijd** aanmaken en/of verwerken
- Mogelijkheden van **instantiëren** via de **constructor Date()**

```
const d = new Date(); // Geeft de datum van vandaag.
const d = new Date(milliseconds); (1584999999999)
const d = new Date(dateString); ('1995-12-17T03:24:00') ('December 17, 1995 03:24:00')
const d = new Date(year, monthIndex, day[, hours, minutes, seconds, milliseconds]); (1995, 11, 17)
```

- **Properties**
  - Date.prototype (eventuele toevoeging van zelfgeschreven eigenschappen aan een Date object)
- **Methods**
  - Bekijk de vele methodes best online via developer.mozilla.org of w3schools of …
- Gebruik Built-in object **Intl** (Internationalization)
  - Toevoeging voor ECMAScript core voor taal-gevoelige functionaliteiten
  - `new Intl.DateTimeFormat('nl-BE',{weekday:'long'}).format(new Date()) // Resultaat: zondag`

Een JavaScript-datum wordt fundamenteel gespecificeerd als het aantal **milli**seconden dat is verstreken sinds het ECMAScript-tijdperk, dat is gedefinieerd als 1 januari 1970 UTC

Math object

07_math.js

- **Ingebouwd object** voor wiskundige constanten en functies
- Werkt met het **Number** type
  - werkt **niet** met **BigInt**!
- Math is **geen constructor**
  - Alle properties en methods zijn **static** d.w.z.
  - Methode wordt **rechtstreeks** op de **klasse** toegepast en niet op een instantie => `Math.Round`
- **Properties**
  - Constanten, bijvoorbeeld `Math.PI (3.14159)`
- **Methods**
  - **Wiskundige** functies (`abs, cos, random, min, max, log, pow, sign, sqrt, tan`...)
  - Opgelet, verschillende browsers kunnen **afwijkend** resultaat geven (precisie)
  - https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Math#static_methods

String object

08_string.js

- **String global object** is een constructor voor strings of een sequentie van karakters.
- **Syntax**
  - **Contructor**
    - `const a = new String('Thomas More')` genereert een nieuw **object String**.
    - `const b = 'Thomas More'` genereert een nieuwe **primitive string**.
    - `const c = String('Thomas More')` genereert een nieuw **primitive string**.
  - **Escape notation**
    - `\' single quote, \\ backslach, \n new line, \r carriage return, \b backspace…`
  - **Long literal strings**
    - Methode 1: gebruik op het einde de **+ operator**
    - Methode 2: gerbuik de **\ backslash** op het einde van de lijn om aan te geven dat de string verder gaat op de volgende lijn.

```
Object: ▼ String
          0: "T"
          1: "h"
          2: "o"
          3: "m"
          4: "a"
          5: "s"
          6: " "
          7: "M"
          8: "o"
          9: "r"
          10: "e"
          length: 11
        ▶ <prototype>: String { "" }
Primitive string:  Thomas More
```

```
let longString = "This is a very long string which needs " +
                 "to wrap across multiple lines because " +
                 "otherwise my code is unreadable."
```

```
let longString = "This is a very long string which needs \
to wrap across multiple lines because \
otherwise my code is unreadable."
```

String object

08_string.js

- **Omschrijving**
  - Character access
    - Benaderen als een array object met de rechte haken => `return 'cat'[1] // returns "a”`
    - Gebruiken de methode charAt() => `return 'cat’.charAt(1) // returns "a”`
  - Comparing strings
    - Gebruik === (case-sensivite), < of > teken

- **Properties**
  - `String.prototype` (eventuele toevoeging van zelfgeschreven eigenschappen aan een String object)
  - `Length` geeft het aantal karakters van de string

- **Methods**
  - functies `(charAt, concat, endsWith, indexOf, lastIndexOf, padEnd, padStart, replace, substr, trim…)`
  - https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/String#instance_properties

Enkele Methods van built-in objects

- Array
  - Array.isArray()
  - …
- Number
  - Number.parseInt()
  - Number.parseFloat()
  - Number.isInteger()
  - …
- Date
  - Date.UTC()
  - Date.now()
  - Date.parse()
  - …
- Math
  - Math.random()
  - Math.round()
  - ...

Oefeningen